

torch.jit.fork#

<https://pytorch.org/docs/stable/generated/torch.jit.fork.html#torch.jit.fork>

Description:

Create an asynchronous task executing `func` and a reference to the value of the result of this execution. `fork` will return immediately, so the return value of `func` may not have been computed yet. To force completion of the task and access the return value invoke `torch.jit.wait` on the Future. `fork` invoked with a `func` which returns `T` is typed as `torch.jit.Future[T]`. `fork` calls can be arbitrarily nested, and may be invoked with positional and keyword arguments. Asynchronous execution will only occur when run in TorchScript. If run in pure python, `fork` will not execute in parallel. `fork` will also not execute in parallel when invoked while tracing, however the `fork` and `wait` calls will be captured in the exported IR Graph. `fork` tasks will execute non-deterministically. We recommend only spawning parallel `fork` tasks for pure functions that do not modify their inputs, module attributes, or global state. `func` (callable or `torch.nn.Module`) – A Python function or `torch.nn.Module` that will be invoked. If executed in TorchScript, it will execute asynchronously, otherwise it will not. Traced invocations of `fork` will be captured in the IR.

Function Signature

```
torch.jit.fork(func, *args, **kwargs)[source]#
```

Parameters

Returns

Return type

Returns:

`torch.jit.Future[T]`

Code Examples

```
import torch
from torch import Tensor

def foo(a: Tensor, b: int) -> Tensor:
    return a + b

def bar(a):
    fut: torch.jit.Future[Tensor] = torch.jit.fork(foo, a, b=2)
    return torch.jit.wait(fut)

script_bar = torch.jit.script(bar)
input = torch.tensor(2)
# only the scripted version executes asynchronously
assert script_bar(input) == bar(input)
# trace is not run asynchronously, but fork is captured in IR
graph = torch.jit.trace(bar, (input,)).graph
assert "fork" in str(graph)
```

```
import torch
from torch import Tensor

class AddMod(torch.nn.Module):
    def forward(self, a: Tensor, b: int):
        return a + b

class Mod(torch.nn.Module):
    def __init__(self) -> None:
        super(self).__init__()
        self.mod = AddMod()

    def forward(self, input):
        fut = torch.jit.fork(self.mod, a, b=2)
        return torch.jit.wait(fut)

input = torch.tensor(2)
mod = Mod()
assert mod(input) == torch.jit.script(mod).forward(input)
```

Notes & Warnings

WARNING

Warning fork tasks will execute non-deterministically. We recommend only spawning parallel fork tasks for pure functions that do not modify their inputs, module attributes, or global state.

Detailed Documentation

Documentation

Create an asynchronous task executing `func` and a reference to the value of the result of this execution.

`fork` will return immediately, so the return value of `func` may not have been computed yet. To force completion of the task and access the return value invoke `torch.jit.wait` on the Future. `fork` invoked with a `func` which returns `T` is typed as `torch.jit.Future[T]`. `fork` calls can be arbitrarily nested, and may be invoked with positional and keyword arguments. Asynchronous execution will only occur when run in TorchScript. If run in pure python, `fork` will not execute in parallel. `fork` will also not execute in parallel when invoked while tracing, however the `fork` and `wait` calls will be captured in the exported IR Graph.

`fork` tasks will execute non-deterministically. We recommend only spawning parallel `fork` tasks for pure functions that do not modify their inputs, module attributes, or global state.

`func` (callable or `torch.nn.Module`) – A Python function or `torch.nn.Module` that will be invoked. If executed in TorchScript, it will execute asynchronously, otherwise it will not. Traced invocations of `fork` will be captured in the IR.

`*args` – arguments to invoke `func` with.

`**kwargs` – arguments to invoke `func` with.

a reference to the execution of `func`. The value `T` can only be accessed by forcing completion of `func` through `torch.jit.wait`.

Example (fork a free function):

Example (fork a module method):